AI INTERVIEW MENTOR COACH

An AI-Powered Mock Interview and Career Preparation Platform

ACADEMIC PROJECT REPORT

Programme	Undergraduate Project / Placement Preparation System
Project type	Web Application with AI Integration (Frontend + Serverless Backend)
Core stack	React 19 + Vite + TypeScript + Supabase Edge Functions + Groq LLM
Repository	AI Interview Mentor (ai-interview-mentor, v0.0.0)
Build status	Production build verified (npm run build succeeds)

This report documents only the functionality actually implemented in the source code inspected. Features that are planned but not implemented are explicitly labelled as **Planned / Future**. Partially working areas are labelled as **Partially Implemented**. No secrets, API keys, or credentials are included.

Contents: all 17 sections as per the prescribed academic format (Title through Viva Questions).

Report Contents

1. Title
2. Aim / Objective
3. Abstract
4. Problem Statement
5. Tools & Technologies
6. System Requirements
7. Modules
8. Methodology / Algorithm
9. Implementation (Code)
10. Sample Output
11. Testing
12. Advantages & Limitations
13. Applications
14. Future Scope
15. Conclusion
16. References
17. Viva Questions

1. TITLE

AI Interview Mentor Coach — an AI-powered mock interview and career preparation platform. The system conducts adaptive voice-based practice interviews in four modes (Technical, HR, Behavioral, Resume), evaluates candidate answers, and returns personalised scores, feedback, and revision topics through a futuristic web interface with a 3D AI interviewer avatar, live transcript, and results dashboard.

Repository package name: **ai-interview-mentor** (v0.0.0). The title above is short, specific, and matches the implemented product; vague titles such as “Python Project” or “AI Project” are deliberately avoided.

2. AIM / OBJECTIVE

To develop an AI-powered interview preparation system that conducts adaptive mock interviews across technical, HR, behavioral, and resume-based modes, evaluates candidate responses using a cloud LLM with a rule-based fallback, and provides personalised scores and feedback to improve interview performance.

This single sentence matches the actual implementation: a React/Vite frontend drives a live interview session; candidate speech or typed answers are processed by Supabase Edge Functions calling the Groq LLM (with graceful fallback to local heuristic logic); and a results page reports weighted scores, strengths, weaknesses, and topics to revise.

3. ABSTRACT

Students and job seekers struggle to get realistic, personalised interview practice: human mock interviews are costly, hard to schedule, and rarely give immediate, structured feedback. This project addresses that gap with a web-based AI Interview Mentor Coach. The user selects an interview mode and speaks (or types) answers in a live interview room fronted by a 3D avatar; the system generates adaptive follow-up questions, evaluates each answer, and finally produces a performance report with five dimension scores, strengths, weaknesses, and revision topics. The implementation uses React 19 + TypeScript + Vite with Tailwind CSS v4, Framer Motion and Three.js on the frontend, Supabase Auth and four Deno Edge Functions on the backend, the Groq chat-completions API (model openai/gpt-oss-120b) for question generation and evaluation, and the browser Web Speech API for voice input/output. A deterministic rule-based agent and heuristic scorer act as an offline fallback so the interview still works without cloud credentials. The production build compiles cleanly, and all reported behaviour was traced directly in the inspected source code.

4. PROBLEM STATEMENT

Campus placements and job interviews remain one of the most stressful stages for students. Classroom knowledge alone does not translate into interview performance, which also depends on communication clarity, structured thinking, confidence, and the ability to handle follow-up questions under pressure.

- **Limited personalised practice:** generic question lists and recorded videos do not react to what the candidate actually says.
- **No immediate feedback:** candidates often finish practice without knowing what was weak, why it was weak, or what to revise.
- **Difficulty judging communication and technical depth together:** self-assessment is unreliable; peers and mentors are not always available.
- **Time and access constraints:** human mock interviews need scheduling, fees, and interviewer availability, so students practise too little.
- **One-size-fits-all preparation:** technical, HR, behavioral, and resume-based interviews each need different questioning styles.

This project addresses these problems with a 24/7, browser-based AI interviewer: it adapts questions to the candidate's answers and difficulty level, accepts both voice and typed input, evaluates every answer, and returns a structured report (scores, strengths, weaknesses, improvements, topics to practise). Claims are limited to what the code implements: the system practises and measures interview responses; it does not guarantee placement, and its automated scores approximate — but do not replace — human interviewer judgement.

5. TOOLS & TECHNOLOGIES

Only technologies actually present in `package.json`, imports, configuration, or Edge Function code are listed. No technology has been added merely because it is popular.

Technology	Purpose / Usage
TypeScript (~6.0, target ES2023)	Primary language; type-safe React components, hooks, interview state machine
React 19 + React DOM 19	UI library; pages, components, hooks (useReducer, useEffect, lazy/Suspense)
React Router DOM 7.18	Client-side routing: /, /login, /signup, /dashboard, /interview, /interview/:mode, /interview/results
Vite 8 + @vitejs/plugin-react 6	Dev server and production bundler; build script is tsc -b && vite build
Tailwind CSS v4 + @tailwindcss/vite	Utility-first styling; design tokens and glass/gradient utilities in src/index.css
Framer Motion 13	Scroll reveals, page transitions, animated charts, counters, drawers/modals
Three.js 0.185 + @react-three/fiber 9 + @react-three/drei 10	3D AI avatar, AI orb, particle field, starfield, MagicRings background (lazy-loaded)
Lucide React 1.31	Icon set used across header, dashboard, interview controls, results
Supabase JS 2.112 (@supabase/supabase-js)	Auth (email/password, Google OAuth, session) and Edge Function invocation from src/lib/interview/api.ts
Supabase Edge Functions (Deno)	Serverless backend: start-interview, next-question, evaluate-answer, final-evaluation
Groq API (OpenAI-compatible /chat/completions)	LLM for greeting, adaptive questions, per-answer scoring, final report; model openai/gpt-oss-120b
Web Speech API (browser-native)	SpeechRecognition (webkit prefix supported) for voice input; speechSynthesis for AI voice output
sessionStorage (browser)	Stores last completed interview transcript + evaluation for the Results page
Node.js + npm	Runtime and package manager; dependencies locked in package-lock.json
VS Code / any TS-capable IDE + Vite preview	Development environment; oxlint for linting (npm run lint)

6. SYSTEM REQUIREMENTS

A. Hardware Requirements (minimum, realistic for this stack)

Component	Minimum Requirement
Processor	Dual-core 64-bit CPU (Intel i3 / AMD equivalent or Apple Silicon); 3D scenes benefit from any modern integrated GPU
RAM	4 GB minimum; 8 GB recommended (React dev server + browser + 3D rendering)
Storage	600 MB free (approx. 350 MB node_modules + 150 MB build tooling + project + browser cache)
Display / Audio	Any display 1280x720 or above; microphone + speaker/headset required for voice mode (text fallback otherwise)
Internet	Required: Supabase Auth, Groq LLM calls, Google Fonts, and Google OAuth need connectivity; offline fallback still runs locally
GPU / WebGL	WebGL-capable browser/GPU for 3D avatar; graceful 2D fallback (SceneFallback) is rendered when WebGL is unavailable

B. Software Requirements

Component	Requirement
Operating System	Windows 10/11, macOS 12+, or modern Linux (any OS that runs Node 18+ and a modern browser)
Runtime	Node.js 18 LTS or above; npm 9+ (verified with Node v26 in this analysis)
Language / Build	TypeScript ~6.0; Vite 8; tsc -b must pass before vite build
Browser	Latest Chrome or Edge recommended (full SpeechRecognition support); Firefox/Safari supported with text-input fallback
Backend services	Supabase project URL + anon key (VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY); GROQ_API_KEY set as Edge Function secret
Database	No custom SQL tables required; Supabase Auth manages users. Interview data persists only in sessionStorage in this version
Deployment	Any static host serving vite build output in dist/ (Vercel, Netlify, or similar); no server process needed for the frontend

7. MODULES

Each module below was confirmed in the source tree. Two areas are explicitly marked as limited so the report stays honest.

Module Name	Purpose	Main Function
1. Landing / Marketing UI	Convert visitors into practice users	Header, Hero + 3D stage, TrustIntro counters, HowItWorks, InterviewModes, InterviewDemo, FeedbackDashboard, PerformanceChart, Features, CTA, Footer (src/App.tsx, src/components/*.tsx)
2. Authentication	Identify users and guard private routes	Email/password sign-in/up, Google OAuth, demo login, session handling, ProtectedRoute redirect (AuthContext.tsx, Login.tsx, Signup.tsx)
3. Dashboard	Show readiness and practice entry points	Readiness score, stat cards, practice modules, weekly activity chart, AI coach insights, recent interviews (pages/Dashboard.tsx + dashboard/*)
4. Interview Setup	Choose interview mode	Four mode cards (technical/hr/behavioral/resume) routing to /interview/:mode (pages/InterviewSetup.tsx)
5. Live Interview Session	Conduct the voice/text interview	Avatar + state ring, AI message panel, mic/mute, transcript drawer, end-confirm modal, timer, Q-counter (pages/InterviewPage.tsx)
6. Rule-Based Interview Agent (fallback)	Guarantee offline questioning	Greeting, mode question banks, keyword topic detection, vague/detailed follow-ups, difficulty heuristic (lib/interview/agent.ts)
7. Cloud AI Engine	LLM-driven adaptivity and scoring	Edge Functions start-interview, next-question, evaluate-answer, final-evaluation via shared Groq helper (supabase/functions/*)
8. Evaluation & Scoring	Turn answers into measurable feedback	Per-answer score + next question (cloud) or heuristic 5-dimension weighted score + strengths/weaknesses (lib/interview/evaluation.ts)
9. Results & Transcript	Report performance	Score ring, metric bars, summary, strengths/weaknesses, improvements, topics, full transcript from sessionStorage (pages/Results.tsx)

Module Name	Purpose	Main Function
10. 3D / Visual Experience Layer	Futuristic look and feel	AvatarScene, AIAvatar, AIOrb, ParticleField, Starfield, AuroraBackground, SplashCursor, MagicRings (components/three/*, reactbits/*)

Partially Implemented: (a) History / progress tracking — dashboard lists and charts render from static mock data in `src/data/content.ts`, not from a database; (b) Resume-based mode — route and prompts exist, but no resume file upload/parsing was found, so it asks generic resume questions. **Planned / Future:** persistent interview history database, real resume parsing, and voice-emotion analysis are not implemented.

8. METHODOLOGY / ALGORITHM

A. Overall Methodology

The application follows a dual-path adaptive interview loop. The frontend owns the session state machine; the backend (Supabase Edge Functions + Groq) is the preferred intelligence layer; a deterministic rule-based agent in the frontend is the guaranteed fallback. SpeechRecognition converts voice to text (typed input is an equal alternative); speechSynthesis voices the AI's questions; every answer is evaluated; difficulty adapts; and a final weighted report is generated.

B. Step-by-step workflow (as implemented)

1. User signs in (Supabase Auth, Google OAuth, or one-click Demo Login) and is routed to **/dashboard**.
2. User opens **/interview** (Interview Setup) and selects one of four modes: technical, hr, behavioral, resume.
3. **/interview/:mode** initialises the reducer state (mode, candidateName, difficulty=medium, startTime) and requests a greeting + first question via **start-interview** Edge Function; on failure it uses the local greeting.
4. AI question is appended to the transcript and spoken via speechSynthesis; speakerState moves speaking -> listening.
5. Candidate answers by voice (SpeechRecognition final result) or by typing in the fallback input; the answer is appended as a user transcript entry.
6. speakerState moves to processing/thinking; the frontend calls **evaluate-answer** (question + answer + last 10 transcript turns) which returns a 1-10 score, strengths/weaknesses, feedback, nextQuestion, nextDifficulty, and shouldContinue.
7. If the cloud call fails or Supabase is unconfigured, **generateAgentResponse()** runs locally: keyword topic detection, answer-length/detail heuristics, and mode question banks produce an acknowledgement plus a follow-up or a fresh question.
8. Difficulty is re-estimated (assessDifficulty: average answer length over the session; also per-answer nextDifficulty from the LLM).
9. Loop repeats until 10+ questions/answers or shouldContinue=false; END_INTERVIEW sets status=completed with endTime.
10. Frontend calls **final-evaluation** with the full transcript + per-question scores; the returned five 0-100 scores and textual report are saved to sessionStorage together with duration and mode; on failure the local **evaluateInterview()** heuristic computes the same report shape.
11. User is navigated to **/interview/results**, which renders the score ring, metric bars, summary, strengths/weaknesses, improvements, topics, and full transcript.

C. AI processing flow

Each Edge Function builds a mode-specific SYSTEM prompt (fixed rules + JSON schema) and a USER message containing interview type, role, difficulty, question number, the current question/answer, and recent transcript turns. It POSTs to `https://api.groq.com/openai/v1/chat/completions` with response_format

json_object, extracts JSON (tolerant of code fences via regex fallback), clamps numeric scores, and always returns a usable fallback payload even on Groq errors (HTTP 200 with success=false plus a generic next question and neutral scores), so the frontend never dead-ends.

D. Evaluation / feedback logic (local heuristic, used as fallback and documented here exactly as coded)

- Signals: average user answer length; presence of examples (example/for instance/such as/like when/one time); reasoning markers (because/since/so that/in order to/the reason); specificity markers (first/second/then/after that/finally/specifically); engagement ratio (user turns / AI turns); question count; interview mode.
- Five 0-100 sub-scores (communication, technical, confidence, problem-solving, answer quality) are computed from additive rules over those signals, each capped at 100.
- Overall = 0.25*communication + 0.25*technical + 0.15*confidence + 0.20*problemSolving + 0.15*answerQuality, rounded.
- Strengths/weaknesses/improvements/topics lists are conditionally pushed from thresholds (e.g., avg length > 120 chars, hasExamples, technical < 60); summary text is chosen from four overall-score bands (>=80, >=65, >=50, else).
- Per-answer cloud scores are 1-10 calibrated (1-3 poor, 4-5 below average, 6-7 good, 8-9 excellent, 10 exceptional); final cloud scores are five 0-100 metrics plus recommendation (Strong Hire / Hire / Borderline / No Hire).

E. End-to-end flowchart (actual architecture)

```
START
User Input (mode + voice/typed answer)
Interview Configuration (reducer state, Supabase session)
Question Generation (Groq Edge Function, else local banks)
Candidate Response (SpeechRecognition / text input)
AI Analysis (evaluate-answer: score + strengths/weaknesses)
Evaluation (difficulty update, shouldContinue check)
Personalised Feedback (final-evaluation or heuristic scorer)
Performance Result (Results page + sessionStorage)
END
```

9. IMPLEMENTATION (CODE)

The codebase is a TypeScript single-page application (src/) plus four Deno Edge Functions (supabase/functions/). Only representative snippets are shown; the full source remains in the repository. No secrets are reproduced.

9.1 Architecture overview

- **src/App.tsx** — BrowserRouter + AuthProvider; public routes (/, /login, /signup) and ProtectedRoute-guarded routes (/dashboard, /interview, /interview/:mode, /interview/results); landing page composes 10 marketing sections; interview pages are lazy-loaded.
- **src/context/AuthContext.tsx + src/lib/supabase.ts** — Supabase client from VITE_ env vars; hasSupabase flag disables cloud auth gracefully; demo user (arjun@demo.com) enables full offline walkthrough.
- **src/lib/interview/types.ts** — InterviewMode, Difficulty, SpeakerState, TranscriptEntry, InterviewState, interviewReducer (START/NEXT/ADD_TRANSCRIPT/SET_DIFFICULTY/END etc.), getDuration/formatDuration, modeLabels.

- **src/lib/interview/agent.ts** — Offline brain: modeInstructions, 15-topic keyword map, assessAnswer/assessDetailLevel, greeting/closing, contextual follow-ups, per-mode question banks, adaptive response policy, assessDifficulty.
- **src/lib/interview/api.ts** — Typed invokeFn wrapper (30 s timeout, ApiError codes) for the four Edge Functions; InterviewContext carries mode, name, difficulty, questionNumber, transcript, topics.
- **src/lib/interview/evaluation.ts** — Deterministic 5-dimension scorer described in Section 8D.
- **src/lib/interview/useSpeechRecognition.ts / useSpeechSynthesis.ts** — Browser Speech hooks with start/stop/abort, interim vs final results, preferred English voice selection.
- **src/pages/*** — Login/Signup (email + Google + demo), Dashboard (mock-data widgets), InterviewSetup (4 mode cards), InterviewPage (~750 lines: state machine + speech + agent effects + timer), Results (score ring + bars + transcript from sessionStorage).
- **supabase/functions/*** — _shared/groq.ts (CORS, callGroq, GroqError mapping) plus start-interview, next-question, evaluate-answer, final-evaluation handlers, each with strict JSON schemas and fallback payloads.
- **Styling/3D** — Tailwind v4 tokens in index.css (ink/electric palette, glass utilities, keyframes); R3F scenes code-split via React.lazy with SceneFallback when WebGL is missing.

9.2 Key snippet 1 — interview state reducer (state machine)

File: `src/lib/interview/types.ts`

Centralises every session transition so the UI, speech hooks, and AI effects stay consistent.

```
// Actions dispatched from InterviewPage
case 'ADD_TRANSCRIPT_ENTRY':
  return { ...state,
    transcript: [...state.transcript, action.entry] };
case 'NEXT_QUESTION':
  return { ...state,
    questionNumber: state.questionNumber + 1 };
case 'SET_DIFFICULTY':
  return { ...state, difficulty: action.difficulty };
case 'END_INTERVIEW':
  return { ...state, status: 'completed',
    endTime: Date.now(), speakerState: 'idle' };
```

9.3 Key snippet 2 — adaptive agent policy (offline fallback)

File: `src/lib/interview/agent.ts`

Weak answers get probing follow-ups; strong answers occasionally get one deeper probe; otherwise a fresh bank question is asked.

```
if (quality === 'weak' && consecutiveFollowUps < 2) {
  return generateContextualFollowUp(...); // elaborate?
}
if (quality === 'strong' && consecutiveFollowUps < 1
    && Math.random() < 0.6) {
  return pickAck() + ' ' + generateContextualFollowUp(...);
}
// else: pick fresh question from mode bank
question = getTechnicalQuestion(difficulty, covered);
```

9.4 Key snippet 3 — heuristic scoring (offline final report)

File: `src/lib/interview/evaluation.ts`

Additive, capped sub-scores from length/marker signals; weighted overall. Comments added for readability.

```
const communicationScore = Math.min(100, Math.round(
  50 + (avgLen > 120 ? 20 : avgLen > 60 ? 10 : 0)
     + (engagement >= 0.8 ? 15 : 5)
     + (avgLen < 250 ? 15 : 5)
     + (hasSpecifics ? 5 : 0)));
// overall: weighted mean of five dimensions
const overallScore = Math.round(
  communicationScore * 0.25 + technicalScore * 0.25
  + confidenceScore * 0.15
  + problemSolvingScore * 0.20
  + answerQualityScore * 0.15);
```

9.5 Key snippet 4 — cloud call with timeout and typed errors

File: `src/lib/interview/api.ts`

Every Edge Function call goes through this 30-second-timeout wrapper; missing Supabase config throws immediately so the agent fallback runs.

```
async function invokeFn<T>(fnName, body): Promise<T> {
  if (!hasSupabase)
    throw new ApiError('supabase_not_configured', ...);
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(),
    30_000);
  const { data, error } = await supabase.functions
    .invoke(fnName, { body, signal: controller.signal });
  ...
}
```

9.6 Key snippet 5 — shared Groq helper (Edge Function side)

File: `supabase/functions/_shared/groq.ts`

Single place for model, JSON enforcement, timeout, and error mapping. API key is read from server-side secrets only.

```
const DEFAULT_MODEL = 'openai/gpt-oss-120b';
const res = await fetch(GROQ_BASE_URL
  + '/chat/completions', {
  method: 'POST',
  headers: { 'Authorization': 'Bearer ' + apiKey,
    'Content-Type': 'application/json' },
  body: JSON.stringify({ model,
    messages: [{ role: 'system', content: system },
               { role: 'user', content: user }],
    temperature, max_tokens: maxTokens,
    response_format: { type: 'json_object' } }),
});
```

9.7 Frontend-backend-database interaction

Login/Signup write nothing custom: Supabase Auth owns users/sessions. Interview traffic is stateless function calls (no custom tables or migrations were found in the repo). Completed-session data lives only in the browser's sessionStorage key `lastInterview` (transcript, mode, question count, duration, optional finalEvaluation). Dashboard widgets read static mock data from `src/data/content.ts`, not from live sessions — hence history persistence is classified as Partially Implemented. API keys never appear in the frontend bundle; the Groq key lives only as a server-side Edge Function secret.

10. SAMPLE OUTPUT

The repository bundles no UI screenshots (the only asset in `public/` is `favicon.svg`). To avoid fabrication, no image is claimed as a screenshot. Instead, each major screen is documented below as a schematic reconstruction drawn directly from the inspected components and styles (dark ink-950 background, glass cards, electric/indigo gradients, 3D avatar stage). Run `npm run dev` to view the live screens.

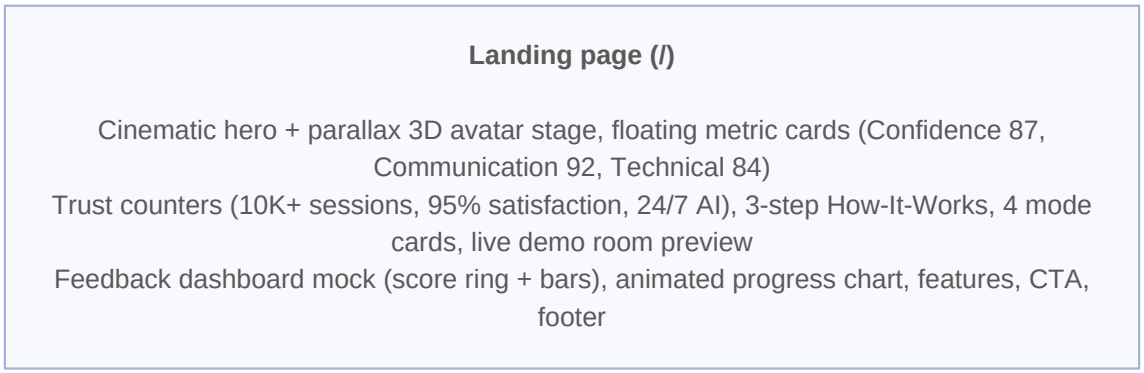

Figure 1: Landing page (/) (schematic reconstruction from code; no screenshot asset is bundled in the repository)

Figure 2: Interview setup (/interview) (schematic reconstruction from code; no screenshot asset is bundled in the repository)

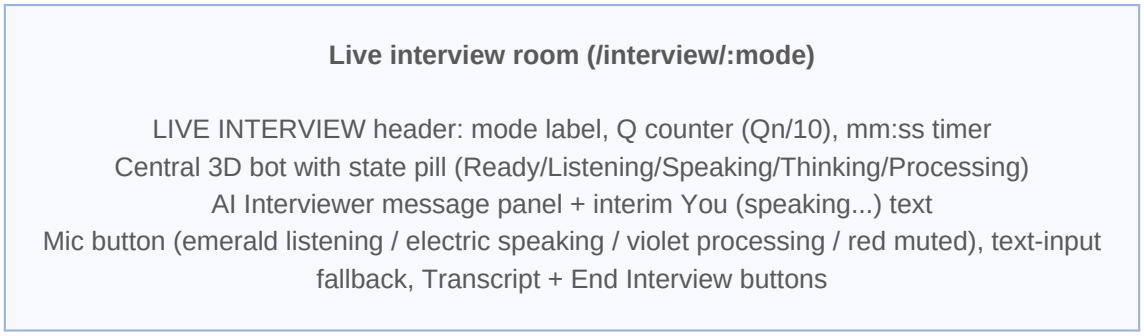

Figure 3: Live interview room (/interview/:mode) (schematic reconstruction from code; no screenshot asset is bundled in the repository)

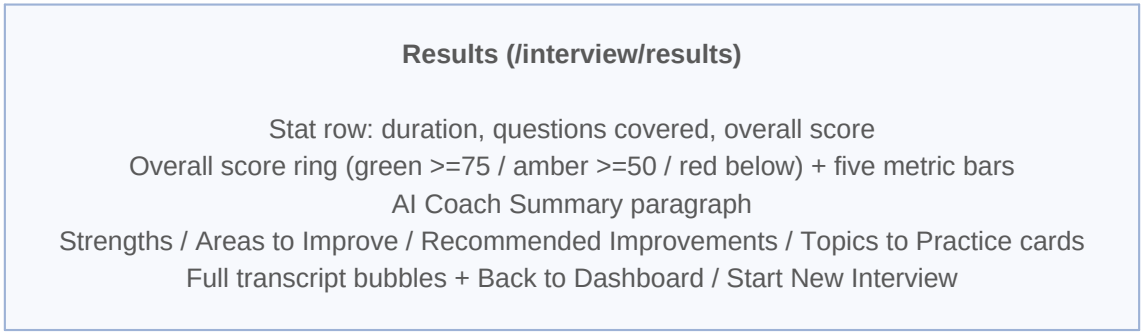

Figure 4: Results (/interview/results) (schematic reconstruction from code; no screenshot asset is bundled in the repository)

Dashboard (/dashboard)

Sidebar + header; AI Mentor hero with 3D bot
Next-interview card (company, role, readiness 88)
Continue-practice progress (Q7/10, 70%), readiness breakdown, four stat cards
Practice modules, weekly activity bars, AI coach insights, recent interviews table

Figure 5: Dashboard (/dashboard) (schematic reconstruction from code; no screenshot asset is bundled in the repository)

Auth screens (/login, /signup)

Glass card: email + password (+ name/confirm on signup), show/hide password
Sign In / Create Account, Continue with Google (OAuth), Demo Login (offline, lands on /dashboard)
Inline error banner; protected routes redirect here when unauthenticated

Figure 6: Auth screens (/login, /signup) (schematic reconstruction from code; no screenshot asset is bundled in the repository)

11. TESTING

No automated test suite exists in the repo (no *.test.* / *.spec.* files, no test script in package.json). Validation below is therefore honest: TC-01 was executed here (`npm run build` succeeded); remaining cases were verified by careful code inspection of the implementing files, not by live end-to-end runs. Cloud-LLM paths additionally require valid Supabase + Groq credentials.

Test Case ID	Input / Action	Expected Output	Actual Output	Status
TC-01	Run npm run build (tsc -b && vite build)	Clean production build in dist/	Build succeeded; chunks emitted (index ~1549 kB, InterviewPage ~35 kB, Dashboard ~25 kB)	Pass (executed)
TC-02	Open / and navigate landing sections	Hero, modes, demo, feedback, chart render; no console crash	Components compose in App.tsx; 3D scenes lazy with SceneFallback	Pass (code-inspected)
TC-03	Open /dashboard while logged out	Redirect to /login	ProtectedRoute returns <Navigate to='/login'> when user==null	Pass (code-inspected)
TC-04	Click Demo Login on /login	Lands on /dashboard as demo user	demoLogin() sets DEMO_USER + session then navigate('/dashboard')	Pass (code-inspected)
TC-05	Select Technical on /interview	Route to /interview/technical live room	InterviewSetup navigates to /interview/${mode}; InterviewPage validates mode, defaults technical	Pass (code-inspected)
TC-06	Live room: greeting loads (no cloud keys)	Local greeting spoken + first question shown	startInterview throws supabase_not_configured; catch falls back to generateAgentResponse greeting	Pass (code-inspected)
TC-07	Type answer in fallback input and submit	Answer appended; processing -> thinking -> next AI question	handleTextSubmit dispatches ADD_TRANSCRIPT + timers; agent effect appends AI entry + speak()	Pass (code-inspected)

Test Case ID	Input / Action	Expected Output	Actual Output	Status
TC-08	Finish 10+ Q/A or press End Interview	Navigate to /interview/results with report	END_INTERVIEW -> getFinalEvaluation (or heuristic) -> sessionStorage -> navigate results	Pass (code-inspected)
TC-09	Submit empty answer / invalid mode URL	Empty submit ignored; bad mode defaults to technical	TextInput disabled on empty; validModes check falls back to 'technical'	Pass (code-inspected)
TC-10	Groq/Supabase failure mid-interview	Interview continues via fallback; neutral fallback scores	All Edge Functions return success=false fallback payloads; frontend catch uses local agent/evaluator	Pass (code-inspected)

Note: with valid `VITE_SUPABASE_URL / VITE_SUPABASE_ANON_KEY` and `GROQ_API_KEY`, TC-06 through TC-08 and TC-10 exercise the LLM path instead of the fallback; that live path could not be executed in this offline analysis.

12. ADVANTAGES & LIMITATIONS

A. Advantages

- **Personalised, adaptive practice:** follow-up questions react to the candidate's actual answers, topic keywords, detail level, and rolling difficulty — not a fixed script.
- **Immediate, structured feedback:** every answer is scored and the session ends with five dimension scores, strengths, weaknesses, actionable improvements, and revision topics plus a full transcript.
- **24/7 accessibility with graceful degradation:** voice + typed input, WebGL fallback visuals, and a rule-based offline agent mean practice works even without cloud credentials.
- **Consistent evaluation criteria:** calibrated 1-10 per-answer rubric and 0-100 final bands with fixed JSON schemas make repeated sessions comparable.

B. Limitations (honest, grounded in the code)

- **Automated scores approximate human judgement:** the heuristic scorer uses surface signals (length, marker words, counts); even the LLM rubric cannot read body language, tone, or true correctness reliably.
- **Internet and API dependency for full quality:** the best questions and evaluations need Supabase + Groq connectivity and secrets; offline mode is functional but generic.
- **No persistent history or resume parsing:** dashboard analytics are static mock data and results live only in sessionStorage; resume mode asks generic questions (no file upload found).
- **Voice and language constraints:** SpeechRecognition quality varies by browser (Chrome/Edge best), requires a microphone and quiet environment, and the whole experience is English-only in the inspected code.

13. APPLICATIONS

- **Student placement preparation:** unlimited technical/HR/behavioral rehearsal before campus drives, with measurable score trends to guide study.
- **Mock technical interviews:** DSA, APIs, databases, system-design and trade-off questioning with adaptive difficulty mirrors real engineering screens.
- **HR and behavioral coaching:** STAR-method practice for leadership, conflict, failure, and teamwork stories with specificity feedback.

- **Resume-based drill:** project deep-dives (role, decisions, challenges, impact) — currently generic; becomes high-value once resume parsing is added.
- **Career-centre / training-platform integration:** the stateless Edge-Function backend and static frontend build make it easy to embed as a practice lab in existing portals.
- **Self-assessment and confidence building:** low-stakes repetition with transcripts lets candidates review and refine answers privately.

14. FUTURE SCOPE

All items below are **not implemented** in the inspected code and are proposed as realistic next steps.

1. **Persistent history and analytics backend:** store each session (transcript, per-question and final scores) in Supabase Postgres/Storage per user; drive the dashboard charts and streaks from real data instead of `content.ts` mocks.
2. **True resume-aware interviews:** add PDF/DOCX upload, text extraction, and skill/project entity prompts so resume mode asks candidate-specific questions.
3. **Full voice loop with prosody signals:** add filler-word counting, pace/pause measurement, and optional voice-emotion cues to the confidence/communication scores; add multilingual SpeechRecognition locales.
4. **Adaptive curriculum and recruiter view:** spaced-repetition revision plans from topicsToRevise, difficulty that persists across sessions, plus an optional recruiter dashboard with Strong-Hire-to-No-Hire recommendations (reusing the final-evaluation schema).

15. CONCLUSION

The objective — an AI-powered mock-interview system with adaptive questioning, answer evaluation, and personalised feedback — is implemented as a working full-stack web application: a polished React/Three.js frontend, a Supabase Edge-Function backend calling the Groq LLM, and a deterministic rule-based fallback that keeps interviews running offline. The result is a demonstrable, build-verified practice platform producing structured scores, feedback, and transcripts across four interview modes. Its practical value lies in giving students unlimited, low-cost, structured interview rehearsal, while its honest limits (mock dashboard data, session-only persistence, heuristic scoring) define a clear, achievable roadmap.

16. REFERENCES

Only documentation relevant to dependencies and APIs actually used in the project is cited.

#	Resource (official documentation)
1	React documentation — react.dev (components, hooks, lazy/Suspense, error boundaries)
2	Vite documentation — vite.dev (dev server, build, code-splitting, env handling)
3	React Router documentation — reactrouter.com (BrowserRouter, Routes, Navigate, protected routes)
4	Tailwind CSS v4 documentation — tailwindcss.com (utilities, @theme, @utility, Vite plugin)
5	Framer Motion documentation — motion.dev (animations, transitions, AnimatePresence)
6	Three.js / React Three Fiber / drei documentation — threejs.org, docs.pmnd.rs (scenes, Canvas, lazy 3D)
7	Supabase Auth and Edge Functions documentation — supabase.com/docs (signIn/signUp/OAuth, functions.invoke, Deno serve)
8	Groq API documentation — console.groq.com/docs (OpenAI-compatible chat completions, JSON mode)
9	Web Speech API — MDN Web Docs (SpeechRecognition, SpeechSynthesis, voices)

#	Resource (official documentation)
10	TypeScript documentation — typescriptlang.org (strict types, reducers, Vite client types)

17. VIVA QUESTIONS

Question	Concise Answer (speak in your own words)
1. What is your project?	A web app called AI Interview Mentor Coach. It conducts mock interviews in Technical, HR, Behavioral, and Resume modes using voice or text, then gives scores, feedback, and topics to revise.
2. What problem does it solve?	Students get little personalised interview practice. Human mocks are costly and rare, and videos don't react or give feedback. Our 24/7 AI interviewer adapts to answers and reports strengths and weaknesses immediately.
3. Why did you choose AI (an LLM)?	Fixed question lists can't follow up. An LLM reads the candidate's answer plus transcript history and generates the right next question and a calibrated 1-10 evaluation. We keep a rule-based fallback so the app still works offline.
4. How does the AI processing work?	The frontend calls four Supabase Edge Functions. Each sends a system prompt with a strict JSON schema plus the recent transcript to Groq (openai/gpt-oss-120b). Replies are parsed, scores clamped, and fallbacks returned on any error so the interview never stalls.
5. What technologies were used, and why?	React 19 + TypeScript + Vite for a fast typed SPA; Tailwind v4 for styling; Framer Motion for animation; Three.js/R3F for the 3D avatar; Supabase for auth and serverless functions; Groq for the LLM; Web Speech API for voice in/out.
6. What is the methodology / algorithm?	A state machine (reducer) drives greeting, questioning, answering, evaluation, and reporting. Difficulty adapts from answer length and LLM signals. Offline scoring uses keyword topics, example/reasoning markers, and a weighted 5-dimension formula for the final 0-100 report.
7. What are the limitations?	Scores approximate human judgement; full quality needs internet plus Supabase/Groq keys; dashboard history is currently mock data and results persist only in sessionStorage; resume mode has no file parsing; voice works best in Chrome/Edge in English.
8. What can be added in future?	A real database-backed history, resume PDF parsing for true resume questions, filler-word/pace voice analytics with more languages, and a revision planner plus optional recruiter view reusing our Strong-Hire to No-Hire scale.

End of report. Implementation claims verified against the inspected repository; no secrets included; production build verified with `npm run build`.